

Leveraging Pillow 12.1.0 for a Stateless Image-Processing SaaS

Context and stateless requirements

The attached brief describes a small, serverless image-processing service designed to run on **AWS Lambda** (primary) and also be packaged as an **EC2 AMI for AWS Marketplace distribution**, with an emphasis on **ephemeral execution** and **statelessness** (no session state between requests, no database, no shared cache). It proposes a typed-schema approach (e.g., request/response models) and an initial functional scope centred on **resize**, **format conversion**, **compression/quality**, **EXIF orientation normalisation**, **thumbnail generation**, and a **metadata endpoint** (dimensions, format, colour mode, optional EXIF summary).

This operating model aligns well with Pillow's strengths: most Pillow operations are **pure, per-request transformations** operating on in-memory data structures, with deterministic outputs when you control parameters (size, resampling, output format, encoder options). The core design challenge is less about "state" and more about **resource safety** (CPU, memory, decompression bombs), **predictable output semantics**, and **build-time codec availability** for the formats you want to support in production.

Pillow 12.1.0 snapshot and production-relevant capabilities

As of **31 January 2026**, the latest Pillow release on PyPI is **12.1.0**, released **2 January 2026**. ¹

Pillow supports a broad range of raster formats. The documentation notes that **over 30 formats** can be identified and read, while write support is "less extensive" but covers "most common" interchange/presentation formats. Importantly for an API service design: `open()` **sniffs by file contents** (not filename), while `save()` typically selects a format based on the output name unless you specify it explicitly. ²

Several behaviours and safeguards matter directly for a stateless SaaS:

- **Lazy decoding:** `Image.open()` identifies the file but defers reading pixel data until you process the image or call `load()`. This is useful for fast metadata inspection, but it also means file handles may remain open if not managed carefully. ³
- **Validation option:** `Image.verify()` checks file integrity without fully decoding pixel data; if you intend to subsequently process the image, you must reopen it after verification. ⁴
- **Decompression bomb protection:** Pillow raises a `DecompressionBombWarning` when pixel count exceeds `MAX_IMAGE_PIXELS`; the threshold can be changed (or disabled), and a `DecompressionBombError` is raised for images exceeding **twice** the limit. ³
- **Incremental parsing:** the `ImageFile.Parser` supports decode "piece by piece" (useful if you ever add streaming uploads). ⁵

Finally, codec support is significantly influenced by **external libraries present at build time**. Pillow documents that you do not need every external library for "basic features", but **zlib and libjpeg are required by default**; other libraries enable additional formats/features (e.g., libwebp, littlecms, libavif).

⁶ A stateless service that aims to be “plug-and-play” for customers (especially on AWS Marketplace AMIs) should treat these dependencies as first-class build artefacts, not incidental details.

Stateless-friendly feature catalogue mapped to Pillow modules

This section focuses on features that are (a) easy to implement in a request/response API, and (b) naturally compatible with stateless execution.

Practical module map for a stateless image API

Stateless API capability (request → response)	Pillow module(s) and selected functions	Notes for SaaS integration	Stateless fit
Decode + identify image	<code>PIL.Image.open()</code>	<code>open()</code> is lazy (pixels read later). Can restrict formats via the <code>formats</code> argument and handle <code>UnidentifiedImageError</code> for unsupported/invalid inputs. ⁷	High
Fast integrity pre-check	<code>Image.verify()</code>	Good for rejecting corrupted images early; requires reopen to proceed with processing. ⁴	High
Resize (bounded box)	<code>Image.thumbnail()</code> or <code>ImageOps.contain()</code>	<code>thumbnail()</code> mutates in place; <code>contain()</code> returns resized image preserving aspect ratio. ⁸	High
Resize-to-fill / cover	<code>ImageOps.cover()</code>	Produces an image that covers the target box while maintaining aspect ratio. ⁹	High
Resize + crop to exact aspect	<code>ImageOps.fit()</code>	Resizes and crops to the requested size; supports crop centring control. ¹⁰	High
Pad to exact size	<code>ImageOps.pad()</code>	Adds padding to reach specified dimensions; useful for “letterbox” thumbnails. ⁸	High
Basic geometry transforms	<code>Image.rotate()</code> , <code>Image.transpose()</code>	<code>transpose()</code> supports 90°/ flip operations. ¹¹	High
EXIF orientation normalisation	<code>ImageOps.exif_transpose()</code>	Applies EXIF Orientation (if not 1) and removes orientation data. ¹²	High

Stateless API capability (request → response)	Pillow module(s) and selected functions	Notes for SaaS integration	Stateless fit
Simple effects & filters	<code>Image.filter()</code> + <code>PIL.ImageFilter.*</code>	Predefined filters (e.g., <code>BLUR</code> , min/max filters) applied via <code>Image.filter()</code> . ¹³	High
Brightness/contrast/colour/sharpness	<code>PIL.ImageEnhance.*</code>	Enhancement classes share <code>.enhance(factor)</code> interface; factor 1.0 returns a copy. ¹⁴	High
Quick LUT-based colour transforms (advanced but still stateless)	<code>PIL.ImageFilter.Color3DLUT</code>	Enables complex colour transformations via precomputed 3D LUTs. ¹⁵	Medium
Colour profile conversion (CMYK→RGB, embed ICC)	<code>PIL.ImageCms</code>	Uses LittleCMS2 for ICC management; supports embedding output ICC profile bytes. ¹⁶	Medium
Metadata extraction (format, mode, EXIF, ICC)	<code>Image.info</code> , <code>Image.getexif()</code> (plus format-specific info keys)	JPEG and PNG loaders expose metadata differently; see format notes below. ¹⁷	High
Codec availability introspection	<code>PIL.features.check()</code> , <code>PIL.features.pilinfo()</code>	Useful for runtime health endpoints ("what formats/codecs are available in this build?"). ¹⁸	High

A key practical constraint: Pillow's `ImageOps` documentation notes it is "somewhat experimental", and that **most operators only work on L and RGB images**. ¹⁹ In a multi-format SaaS context (PNG with alpha, palette images, CMYK JPEGs), it is usually safest to normalise with `convert("RGB")` / `convert("RGBA")` (depending on your alpha support policy) prior to applying many `ImageOps` operations.

"Easy wins" aligned to the brief's initial scope

The brief's initial operations—resize, format conversion (PNG/JPEG/WebP), compression/quality changes, EXIF orientation normalisation, thumbnails, and metadata—are all directly supported by the above primitives. The most straightforward implementations rely on:

- `Image.open()` plus strict validation and resource limits ³
- `ImageOps.exif_transpose()` before geometry operations (normalises orientation early, avoids "sideways thumbnail" issues) ¹²
- `ImageOps.contain/fit/pad` for predictable thumbnail policies, instead of ad-hoc crop maths ⁹

- `Image.save()` with carefully chosen format-specific options (next section), returning bytes to the client rather than persisting artefacts server-side ²⁰

Formats, conversions, and metadata handling with Pillow

A stateless image-processing SaaS typically benefits from supporting a small set of widely used output formats with high-quality encoders and clear semantics. Pillow can do this, but each format has different knobs and metadata behaviour.

Encoding and conversion options to expose safely

The table below collects the most practical encoder options for a stateless API surface where you want “good defaults” plus a small number of explicit parameters.

Output format	Pillow notes (support + key save options)	Suggested stateless API parameters	Metadata handling implications
JPEG	Reads JPEG/JFIF/Adobe JPEG; writes standard and progressive JFIF. Save options include <code>quality</code> (0–95 or <code>keep</code>, default 75), <code>optimize</code>, <code>progressive</code>, <code>icc_profile</code>, <code>exif</code>, <code>keep_rgb</code>, <code>subsampling</code>. Values above 95 are discouraged; 100 disables parts of compression and yields large files. ²¹	<code>quality</code> (int or “keep”), <code>progressive</code> (bool), <code>optimize</code> (bool), <code>subsampling</code> (enum), optional <code>keep_rgb</code> (bool)	JPEG <code>info</code> may include ICC profile and raw EXIF; you must pass <code>icc_profile=</code> / <code>exif=</code> to preserve. If you omit them, output is saved with no profile/EXIF. ²²
PNG	Reads/writes PNG in many modes. EXIF can be read from PNG, but not guaranteed in <code>info</code> until <code>load()</code> is called. Save options include <code>optimize</code>, <code>compress_level</code> (0–9, default 6), <code>pnginfo</code> (custom chunks), <code>icc_profile</code>, <code>exif</code>. When <code>optimize=True</code>, <code>compress_level</code> is effectively forced to 9. PNG text chunks are subject to per-chunk and total limits to prevent decompression bombs. ²³	<code>optimize</code> (bool), <code>compress_level</code> (int), optional <code>preserve_text_chunks</code> (bool), optional <code>strip_metadata</code> (bool)	Decide whether to preserve text chunks; be aware Pillow limits decompressed PNG text chunk sizes and total memory for safety. ²⁴

Output format	Pillow notes (support + key save options)	Suggested stateless API parameters	Metadata handling implications
WebP	Reads and writes WebP; requires libwebp v0.5.0+. Save options include <code>lossless</code>, <code>quality</code> (0-100, default 80), <code>alpha_quality</code>, <code>method</code> (0-6), <code>exact</code>, and can embed <code>icc_profile</code>, <code>exif</code>, <code>xmp</code>. ²⁵	<code>lossless</code> (bool), <code>quality</code> (int), <code>method</code> (int), optional <code>alpha_quality</code> (int)	Straightforward “strip vs preserve” policy: only embed ICC/EXIF/XMP if explicitly requested (or if your compliance policy requires preserving). ²⁶
AVIF	Reads/writes AVIF. Pillow exposes save options including <code>quality</code>, <code>subsampling</code>, <code>speed</code>, <code>max_threads</code>, <code>codec</code> and can include <code>icc_profile</code>, <code>exif</code>, <code>xmp</code>. However: only 8-bit AVIF can be saved, and all AVIF images are decoded as 8-bit RGB(A). ²⁷	<code>quality</code> (int), <code>speed</code> (int), optional <code>max_threads</code> (int), optional <code>codec</code> (enum)	AVIF is attractive for modern delivery; you may want a “format=auto” option at the API level (if your clients can negotiate), but ensure your build includes libavif (and codecs) reliably. ²⁸

Two practical metadata patterns emerge in the documentation:

- For **JPEG**, Pillow surfaces `info` keys such as `icc_profile` and raw `exif`, and saving requires explicitly passing metadata back in if you want to preserve it. ²²
- For **PNG**, EXIF reading exists but is delayed until `load()`, and PNG text chunks are bounded for safety. ²⁴

For a SaaS product, this suggests a clean contract: make metadata handling explicit (e.g., `metadata_policy = strip | preserve | preserve_safe_subset`), and default to **strip** for privacy unless the target use case requires preservation (e.g., professional photography workflows).

Build-time codec availability and runtime detection

If you plan to offer WebP and AVIF output reliably, you need to treat Pillow’s external libraries as required build artefacts. Pillow’s build documentation highlights the role of libraries such as **libwebp** (WebP format) and **libavif** (AVIF) and notes that Pillow requires **libavif ≥ 1.0.0**, while libavif itself depends on an AVIF encoder/decoder implementation (e.g., dav1d, rav1e, libaom). ⁶

At runtime, `PIL.features.check()` and `PIL.features.pilinfo()` give you a robust way to expose a “capability endpoint” (or internal readiness checks) so you can confirm which codecs and features are present in the deployed artefact. ¹⁸

Stateless service integration patterns, safety guards, and operational fit

Stateless pipeline diagram

A stateless request/response image transform is naturally expressed as a pure pipeline:

```
HTTP request (image bytes + typed transformation spec)
→ validate (size, content-type, allowed formats)
→ decode (Image.open) + safety checks
→ normalise (e.g., exif_transpose, mode convert)
→ transform (resize/crop/filter/colour)
→ encode (save to chosen format + encoder options)
→ HTTP response (bytes + selected metadata)
```

Each stage can be implemented without storing state across requests; all necessary context travels in the request payload.

Safety controls strongly recommended for untrusted uploads

A SaaS endpoint that accepts arbitrary images tends to be exposed to malformed files and intentional resource exhaustion attempts. Pillow provides several hooks you can leverage:

- **Decompression bomb limits:** enforce a maximum megapixel threshold via `MAX_IMAGE_PIXELS`, and consider treating `DecompressionBombWarning` as an error for strict SaaS environments. The documentation explicitly describes warning vs error behaviour (error at $> 2\times$ the limit). ³
- **Truncated images:** by default, Pillow does not load truncated JPEG/PNG; it documents `ImageFile.LOAD_TRUNCATED_IMAGES` as an override. For a SaaS, it is usually safer to keep this `False` unless you have a strong product reason to accept damaged inputs. ²⁹
- **Block-based processing:** `ImageFile.MAXBLOCK` exists specifically because Pillow processes data in blocks to reduce resource spikes; it is user-configurable, but the default behaviour is already aligned with controlled resource usage. ³⁰
- **Metadata-related limits:** PNG text chunks are bounded (per chunk and in aggregate) “to prevent decompression bombs.” If you expose “preserve PNG text metadata”, this limit should inform your API constraints and documentation. ²⁴
- **Format allow-lists:** because `Image.open()` can accept an optional `formats` list/tuple, you can restrict the set of formats checked and reduce risk/complexity. ³

Performance and determinism considerations for stateless workloads

For the brief’s initial scope, the highest-value optimisations tend to be:

- **Normalise early:** apply `ImageOps.exif_transpose()` before resizing and thumbnail generation. This prevents inconsistent geometry results and removes the orientation tag as documented. ¹²
- **Use format-aware load shortcuts:** for JPEG downscales, `draft()` can speed processing by converting and loading at reduced resolution (1/2, 1/4, 1/8) when appropriate. This is especially attractive for thumbnail endpoints. ³¹

- **Constrain AVIF/WebP effort:** AVIF exposes `speed` and `max_threads`, and WebP exposes `method`; these are natural “quality vs latency” controls for serverless workloads. ³²
- **Avoid silent mode-dependent behaviour:** since ImageOps operators are often `L` / `RGB` oriented, a consistent mode normalisation policy (e.g., always output RGB unless explicitly asked) reduces edge cases. ³³

Market survey of comparable SaaS image-processing platforms

The market clusters into two broad categories:

- **Full media platforms** bundling transformation, storage, asset management, and CDN delivery.
- **Narrow APIs** specialising in optimisation/compression or pay-per-operation transforms.

The table below focuses on vendors whose public pricing pages clearly document features and limits relevant to benchmarking a stateless image-processing API offering.

Competitor pricing and limits

All prices below are shown as published on vendor sites (typically USD) and may exclude taxes.

Provider	Positioning (as communicated on pricing pages)	Pricing model and entry tiers	Notable limits/units relevant to benchmarking
<code>entity["company","Cloudinary","media management saas"]</code>	Image & video APIs with transformations, upload tooling, and CDN delivery; credits-based plans. ³⁴	Free plan (\$0) and paid tiers. “Plus” is \$99/month and includes 225 monthly credits ; “Advanced” is \$249/month and includes 600 monthly credits . ³⁵	Units are expressed in credits ; free plan indicates 25 monthly credits . ³⁵
<code>entity["company","imgix","image rendering cdn"]</code>	Image rendering API with transformation at the edge; plans include unlimited transformations (self-service). ³⁶	Self-service: Free (\$0) up to 1,000 origin images ; Basic \$75/month for 5,000 origin images ; Growth \$300/month for 25,000 origin images . ³⁶	Explicitly states “Unlimited image transformations” in self-service features. ³⁶

Provider	Positioning (as communicated on pricing pages)	Pricing model and entry tiers	Notable limits/units relevant to benchmarking
<code>entity["company","ImageKit","media optimisation platform"]</code>	Media optimisation + transformations + storage (DAM), with bandwidth/storage-based tiers; emphasises AVIF optimisation on higher tiers. ³⁷	Paid tiers include Lite \$9/month and Pro \$89/month , with bandwidth and storage inclusions/overages. ³⁷	Pro tier lists "Automatic AVIF optimization available on-demand". ³⁸
<code>entity["company","Uploadcare","file upload and image cdn"]</code>	Operations-based file uploading + image optimisation/transformation + CDN delivery; explicitly includes EXIF metadata removal and ICC profile handling. ³⁹	Free (\$0) includes 1,000 operations/month . Pro is \$66/month with 100,000 operations/month ; Business is \$166/month with 250,000 operations/month . ⁴⁰	Overages: Pro includes \$0.5 per 1,000 operations beyond included. Traffic and storage are metered with included quotas and per-GB pricing. ⁴¹
<code>entity["company","Filestack","file handling saas"]</code>	File handling with image transformations and conversions; transformation results cached for 30 days for counting purposes. ⁴²	Plan Start \$69/month ; Grow \$199/month ; Scale \$379/month . ⁴³	Transformations are quota-based; REST API transformations create a cached version active for 30 days and "only be counted as one conversion" during that period. ⁴²

Provider	Positioning (as communicated on pricing pages)	Pricing model and entry tiers	Notable limits/units relevant to benchmarking
<code>entity["company","Cloudflare","internet infrastructure company"]</code>	CDN-linked image transformation and (optionally) storage; pricing hinges on <i>unique</i> transformations, stored images, and delivered images. ⁴⁴	Free plan includes 5,000 unique transformations/month . Paid pricing: first 5,000 included + \$0.50 per 1,000 unique transformations/month . Storage: \$5 per 100,000 images stored/month . Delivery: \$1 per 100,000 images delivered/month . ⁴⁴	Unique transformations are counted over a 30-day sliding window , and “format=auto” can count as a single billable transformation even if served as different formats. ⁴⁵
<code>entity["company","Tinify","image compression api"]</code>	Developer API focused on compress/convert/resize with pay-per-compression pricing; strongly comparable to a “single-purpose” stateless API. ⁴⁶	First 500 compressions/month free ; next 9,500 at \$0.009 per image ; after 10,000 at \$0.002 per image . ⁴⁶	Pure per-unit pricing provides a clear benchmark for “per-transform” SaaS pricing. ⁴⁶
<code>entity["company","Kraken.io","image optimisation api"]</code>	Image optimisation/compression with API and quota-tiered subscriptions. ⁴⁷	Paid tiers start at \$5/month (Micro: 500 MB images/month), then \$9/month (2 GB), \$19/month (5 GB), etc. Free account includes 100 MB testing quota . ⁴⁷	Metered in total input image volume (GB/month) with additional GB charges per tier. ⁴⁷

Pricing model pattern observed across the market

Even though these services differ, their published pricing pages repeatedly emphasise one of three measurable “units”:

1. **Per transformation / per unique derivative** (e.g., unique transformations with a time window)

⁴⁸

2. **Per operation/credit** (bundling transformations + delivery/storage operations) ⁴⁹
3. **Per compressed image / per GB processed** (optimisation-focused providers) ⁵⁰

A notable and strategically important pattern is the emphasis on *uniqueness and caching windows* for transformation counting (Cloudflare's 30-day sliding window; Filestack's 30-day cached derivative semantics). This reduces repeated billing for the same derivative and makes costs more predictable for customers. ⁴⁸

For a stateless service that explicitly avoids server-side caches, the commercial implication is that customers may experience higher costs for repeated transformations unless you (a) encourage downstream caching (e.g., via CDN headers) outside your service boundary, or (b) adopt a pricing metric that approximates uniqueness from the client perspective (for example, charging less for idempotent repeated requests if the client supplies a stable derivative key or hash). This is an inference based on competitor pricing mechanics, not a property of Pillow itself. ⁴⁸

Recommendations for feature prioritisation and integration

Highest-value features to implement first

Based on (i) your brief's initial scope, (ii) features commonly highlighted by comparable SaaS providers (resize/crop, conversion/compress, metadata handling), and (iii) Pillow's most straightforward APIs, the following sequence is a pragmatic starting roadmap.

Priority band	Feature set	Why this should be prioritised	Pillow building blocks (examples)
Core	Orientation-safe resize + thumbnail policies; JPEG/PNG/WebP output with quality; metadata endpoint	Matches the initial scope and aligns with ubiquitous market demand (nearly every platform offers resize + conversion/compress). ⁵¹	<code>Image.open()</code> + limits ³ ; <code>ImageOps.exif_transpose()</code> ¹² ; <code>ImageOps.contain/fit/pad</code> ⁹ ; <code>save()</code> options per format ⁵²
Near-term expansion	"Safe effects pack": blur/sharpen, brightness/contrast/colour, grayscale; PNG text-handling policies	Many platforms advertise filters/adjustments; easy to implement and differentiate without state. ⁵³	<code>Image.filter(ImageFilter.*)</code> ¹³ ; <code>ImageEnhance.*.enhance()</code> ¹⁴ ; <code>ImageOps.grayscale/autocontrast</code> (with mode normalisation) ⁵⁴

Priority band	Feature set	Why this should be prioritised	Pillow building blocks (examples)
Capability differentiators	AVIF output (plus format-aware defaults); ICC profile conversion (CMYK→RGB)	AVIF is explicitly marketed by several competitors; ICC handling is mentioned as a serious feature (professional or print workflows). 55	AVIF save options + build with libavif 27 ; <code>ImageCms</code> colour management 16

Practical packaging guidance for AWS deployment artefacts (Lambda and AMIs)

Because format support depends on external libraries, the most important operational recommendation is to make codec support explicit and testable:

- Define a “supported formats” contract (inputs + outputs) and validate it using `PIL.features.pilinfo()` / `check()` in CI and at runtime. 18
- If offering AVIF, ensure your build includes libavif $\geq 1.0.0$ and appropriate codecs, as described in Pillow’s build documentation. 56
- For WebP, ensure libwebp is available (Pillow requires libwebp v0.5.0+). 57
- Keep security defaults strict: leave `ImageFile.LOAD_TRUNCATED_IMAGES = False` unless your product explicitly targets recovery of damaged images. 58
- Treat decompression-bomb controls as part of your public API limits (maximum megapixels accepted), aligning with Pillow’s `MAX_IMAGE_PIXELS` behaviour. 3

Pricing strategy implications drawn from market signals

Competitor pricing suggests two viable benchmarking anchors for a Pillow-backed stateless API:

- **Per-operation tactical benchmark:** Cloudflare’s paid transformation metric is \$0.50 per 1,000 unique transformations after a free allowance. 45
- **Per-image compression benchmark:** Tinify charges \$0.009 per image for the tier immediately after the free allowance (then \$0.002 at higher volumes). 46
- **Hybrid subscription + overages benchmark:** Uploadcare publishes included monthly operations with per-1,000 overage pricing, plus bandwidth/storage metering. 41

A stateless service *without bundled CDN delivery and storage* will often need to price below “full platform” offerings (credits, DAM, delivery, analytics) to be competitive, but above bare-metal costs to cover CPU-heavy transforms and abuse mitigation. This is an inference from how platforms bundle features and meter usage; your actual pricing should be validated by pilots and cost modelling. 59

Primary sources referenced

- Pillow 12.1.0 release metadata (PyPI) 1
- Pillow documentation: Image module, ImageOps, ImageFile, features, Image file formats (JPEG/PNG/WebP/AVIF), ImageFilter, ImageEnhance, ImageCms 60

- Official competitor pricing pages: Cloudinary, imgix, ImageKit, Uploadcare, Filestack, Cloudflare Images, Tinify, Kraken.io 61
-

1 <https://pypi.org/project/pillow/>

<https://pypi.org/project/pillow/>

2 17 20 21 22 23 24 25 26 27 29 31 32 52 57 <https://pillow.readthedocs.io/en/stable/handbook/image-file-formats.html>

<https://pillow.readthedocs.io/en/stable/handbook/image-file-formats.html>

3 4 7 11 60 <https://pillow.readthedocs.io/en/stable/reference/Image.html>

<https://pillow.readthedocs.io/en/stable/reference/Image.html>

5 30 58 <https://pillow.readthedocs.io/en/stable/reference/ImageFile.html>

<https://pillow.readthedocs.io/en/stable/reference/ImageFile.html>

6 28 56 <https://pillow.readthedocs.io/en/stable/installation/building-from-source.html>

<https://pillow.readthedocs.io/en/stable/installation/building-from-source.html>

8 9 10 12 19 33 54 <https://pillow.readthedocs.io/en/stable/reference/ImageOps.html>

<https://pillow.readthedocs.io/en/stable/reference/ImageOps.html>

13 15 <https://pillow.readthedocs.io/en/stable/reference/ImageFilter.html>

<https://pillow.readthedocs.io/en/stable/reference/ImageFilter.html>

14 <https://pillow.readthedocs.io/en/stable/reference/ImageEnhance.html>

<https://pillow.readthedocs.io/en/stable/reference/ImageEnhance.html>

16 <https://pillow.readthedocs.io/en/stable/reference/ImageCms.html>

<https://pillow.readthedocs.io/en/stable/reference/ImageCms.html>

18 <https://pillow.readthedocs.io/en/stable/reference/features.html>

<https://pillow.readthedocs.io/en/stable/reference/features.html>

34 35 59 61 <https://cloudinary.com/pricing>

<https://cloudinary.com/pricing>

36 51 <https://www.imgix.com/pricing>

<https://www.imgix.com/pricing>

37 38 55 <https://imagekit.io/plans/>

<https://imagekit.io/plans/>

39 40 41 49 <https://uploadcare.com/pricing/>

<https://uploadcare.com/pricing/>

42 43 53 <https://www.filestack.com/pricing/>

<https://www.filestack.com/pricing/>

44 45 48 <https://developers.cloudflare.com/images/pricing/>

<https://developers.cloudflare.com/images/pricing/>

46 50 <https://tinify.com/pricing/api>

<https://tinify.com/pricing/api>

47 <https://kraken.io/pricing>

<https://kraken.io/pricing>